Desarrollo de ALERTA MUJER: Sistema de alertas y asistencia para la seguridad de las mujeres basado en una arquitectura modular

José Sebastián Suarez Cumaco, Michael Stiven Sánchez Castañeda, Justin Daniela Bahamon Baloco

Aprendices del SENA — Centro de la Industria, la Empresa y los Servicios – Regional Huila, Colombia

***Resumen*—**Este trabajo describe el desarrollo de Alerta Mujer, un sistema creado para apoyar la seguridad de las mujeres mediante herramientas que permiten enviar alertas de emergencia. El proyecto cuenta con una aplicación móvil para Android y una plataforma web, donde las usuarias pueden activar alertas, compartir su ubicación con contactos de confianza, avisar si no llegan a su destino y consultar recursos de asistencia. El backend se organiza en tres microservicios con arquitectura hexagonal (autenticación, alertas y evidencias) que comparten una base de datos PostgreSQL. Durante el desarrollo se realizaron procesos de análisis, diseño, implementación y pruebas: el código cuenta con 553 pruebas automatizadas aprobadas el 27/10/2026 (351 del backend, 39 de la web y 163 del móvil), que GitHub Actions repite en cada cambio. La eficacia del sistema en emergencias reales no se ha evaluado; el artículo separa lo implementado de lo verificado y de lo que sigue pendiente.

***Index Terms*—alertas de emergencia, seguridad de las mujeres, microservicios, aplicación móvil, plataforma web, Spring Boot, React Native, Angular, JWT**

I. Introducción

A. La violencia de género en Colombia

La violencia de género sigue siendo un problema persistente en Colombia. Durante 2023, el sistema de vigilancia en salud pública recibió 189.540 notificaciones de casos sospechosos de violencia de género e intrafamiliar y de ataques con agentes químicos. Después de excluir registros repetidos, de otros periodos o que no cumplían la definición de caso, el Instituto Nacional de Salud (INS) analizó 158.313 casos, y el 78,4 % (124.127) ocurrió en mujeres [1]. Se trata de casos sospechosos notificados por los servicios de salud, no de hechos confirmados, y solo reflejan las situaciones que llegaron a esos servicios.

Otras fuentes miden aspectos distintos del problema. En la presentación de la tercera edición del estudio Mujeres y hombres: brechas de género en Colombia, ONU Mujeres y el DANE señalan que, según las valoraciones médico-legales de 2023, cada día fueron asesinadas en promedio tres mujeres y 101 resultaron lesionadas por su pareja [2]. Estas cifras provienen de un registro diferente al del INS, por lo que no se suman ni se comparan de forma directa.

Cuando una persona está en riesgo, pedir ayuda depende casi siempre de ella: llamar a una línea de atención, buscar el número de una institución o avisarle a alguien de confianza. Esa información suele estar repartida en canales distintos, y encontrarla en un momento de tensión puede ser difícil. De ahí surge la idea de configurar con anticipación una red de contactos de apoyo y de tener a mano los números de emergencia.

B. Limitaciones de las soluciones tecnológicas existentes

Las aplicaciones móviles de seguridad personal intentan cubrir esa necesidad, pero hay poca evidencia sobre su funcionamiento. En una revisión sistemática de aplicaciones para la prevención de la violencia doméstica, Sumra et al. [3] encontraron que, en las aplicaciones analizadas, la alerta dependía de una acción manual y que no existía una herramienta estandarizada para evaluarlas. Ford et al. [4] estudiaron las aplicaciones disponibles en el Reino Unido y las reseñas de sus usuarios, y documentaron quejas por fallas, por funciones que no hacían lo prometido y por solicitudes de datos personales que se consideraban innecesarias. La sección II amplía estos y otros trabajos.

C. Motivación y propuesta

ALERTA MUJER es un proyecto formativo del SENA, Regional Huila, que busca reunir en una misma solución la activación de alertas, la gestión de contactos de confianza y el acceso a recursos de asistencia. Aunque su nombre alude a las mujeres, la aplicación puede usarla cualquier persona; el enfoque prioritario sigue siendo la prevención de situaciones de riesgo que afectan a las mujeres. El sistema está en desarrollo, no reemplaza los servicios oficiales de emergencia y no garantiza que una alerta sea recibida o atendida.

La solución se compone de una aplicación móvil para Android, una plataforma web y un backend con su base de datos. La plataforma web tiene tres espacios: un sitio público, un portal para usuarios autenticados y un panel administrativo, con acceso controlado por rol. El backend, desarrollado con Java y Spring Boot, se organiza en tres microservicios (Auth, Alert y Evidence) con arquitectura hexagonal, que comparten una base de datos PostgreSQL.

Hoy están implementadas en el código las funciones principales: la activación de la alerta con el botón, desde un contacto, agitando el teléfono o desde el widget; la ventana de cinco segundos para cancelar, con PIN o con el botón de volumen; un PIN de emergencia para cuando obligan a la usuaria a cancelar; el aviso a los contactos por correo y por notificación push, que suena como alarma en el teléfono del contacto y se reenvía una vez si nadie lo abre en cinco minutos;

el seguimiento público por enlace; la grabación de audio y video; el trayecto «Avísame si no llego», y una protección automática que acompaña a la usuaria al salir de sus lugares frecuentes. El módulo de asistencia ofrece tres canales (123, línea púrpura y línea de salud mental): al seleccionar uno, se abre el marcador del teléfono con el número ya ingresado y la persona usuaria decide si hace la llamada. Ninguna de estas funciones se ha verificado todavía con casos de prueba documentados.

D. Objetivos y aporte

El objetivo general del proyecto es desarrollar una solución tecnológica integrada, compuesta por una aplicación móvil Android y una plataforma web, que facilite la activación de alertas de emergencia, la gestión de contactos de confianza y el acceso a información relacionada con situaciones de riesgo. Para ello se plantearon como objetivos específicos documentar los requisitos funcionales y no funcionales; diseñar la arquitectura, la base de datos y las interfaces; desarrollar e integrar los componentes de la aplicación móvil, la plataforma web y el backend; realizar pruebas funcionales y de integración, y elaborar la documentación técnica, de instalación y de usuario.

Este artículo describe el diseño de ALERTA MUJER y su estado de implementación, y separa de forma explícita lo implementado, lo pendiente de verificación y lo que sigue sin construir. Su aporte consiste en documentar una solución que une aplicación móvil, portal web y panel administrativo sobre un backend modular, y en relacionar sus decisiones de diseño con las limitaciones que reporta la literatura. La eficacia del sistema en emergencias reales no se ha evaluado y queda fuera del alcance de este trabajo.

E. Organización del artículo

El resto del artículo se organiza así. La sección II revisa los trabajos relacionados. La sección III presenta la metodología y la sección IV, la implementación. La sección V describe la estrategia de pruebas y validación, y la sección VI, los resultados. La sección VII discute los hallazgos y la sección VIII presenta las conclusiones y el trabajo futuro.

II. Trabajos Relacionados

A. Aplicaciones móviles de seguridad personal

Las aplicaciones de seguridad personal abundan en las tiendas móviles, pero hay poca evidencia sobre su funcionamiento. Sumra et al. [3] identificaron 621 aplicaciones relacionadas con la prevención de la violencia doméstica publicadas entre 2012 y 2022, revisaron 136 y analizaron 20 en detalle. La categoría más numerosa fue la de asistencia en emergencias (44,9 %). En las aplicaciones analizadas, la alerta dependía de una acción manual de la usuaria y ninguna automatizaba su activación. Los autores tampoco encontraron una herramienta estandarizada de evaluación, por lo que trabajaron con descripciones de los desarrolladores, artículos, blogs y reseñas.

Ford et al. [4] llegaron a un diagnóstico parecido a partir de otra fuente: las opiniones de quienes usan estas aplicaciones. De 503 aplicaciones disponibles en el Reino Unido, 86 cumplieron sus criterios de revisión, y de 61 de ellas extrajeron 3.820 reseñas. Aunque el 52,4 % tuvo un tono positivo, las personas usuarias señalaron aplicaciones poco confiables, que no hacían lo que prometían o que tenían funciones que otros podían usar en su contra. Los autores advierten que, sin regulación ni acreditación, quienes están en riesgo pueden terminar dependiendo de tecnología que nunca fue probada.

Las propuestas académicas de aplicaciones concretas se encuentran en un punto similar. SAM [5] es un diseño de interfaz para mujeres en situación de riesgo en Huajuapan de León (Oaxaca, México), desarrollado con un enfoque centrado en el usuario; incluye botones de ayuda, llamadas simuladas, seguimiento y ubicación en tiempo real, y se presenta como prototipo. Lakshmipathy y Kumar [6] describen una aplicación que combina seguimiento de ubicación y contactos de emergencia con recursos legales, clases de defensa personal y apoyo comunitario, diseñada a partir de entrevistas, encuestas y un análisis de las aplicaciones existentes. Ninguno de los dos resúmenes reporta una evaluación en situaciones de emergencia.

ALERTA MUJER coincide con estas soluciones en lo esencial (alerta, contactos de confianza y ubicación) y comparte también su principal debilidad: la falta de validación en condiciones reales. En la versión actual, según la auditoría técnica v4 y el SRS v2.6, las funciones principales ya están en el código: el botón SOS, el widget, la alerta desde la sección de contactos, la activación al agitar el teléfono (un módulo nativo en Kotlin que funciona con la app cerrada), la ventana de cinco segundos para cancelar, los PIN de emergencia y de cancelación, el aviso a los contactos por correo y notificación push con un enlace de seguimiento, el trayecto «Avísame si no llego» y la protección automática al salir de los lugares frecuentes. Lo que falta es verificarlas. Ningún requisito tiene todavía el estado «Verificada», y las pruebas manuales de las funciones nativas y de las notificaciones entre dos teléfonos (PM-04 a PM-09) siguen pendientes.

La activación por movimiento y la protección automática responden a la ausencia de activación automática que señalan Sumra et al. [3]; ya están implementadas, pero solo podrán considerarse un aporte cuando se prueben en dispositivos físicos. Las quejas que recogen Ford et al. [4] también orientan una regla del diseño: la aplicación no debe mostrar como entregada una alerta cuya comunicación no se haya confirmado. La Tabla I resume las características reportadas en cada trabajo.

Tabla I
Comparación de trabajos relacionados según las características reportadas

Trabajo	Funciones o hallazgos reportados	Evaluación reportada
Sumra et al. [3]. Revisión sistemática (621 apps identificadas, 136 revisadas, 20 analizadas)	44,9 % de asistencia en emergencias; activación manual; privacidad como preocupación central	Sin herramienta estandarizada
Ford et al. [4]. Revisión en tiendas del Reino Unido (86 apps, 3.820 reseñas)	86,0 % solicita ubicación y 61,6 % contactos; reseñas sobre fallas y posibles usos indebidos	Análisis de sentimiento y temático de reseñas

Trabajo	Funciones o hallazgos reportados	Evaluación reportada
SAM [5]. Diseño de interfaz (México)	Botones de ayuda, llamadas simuladas, seguimiento, ubicación en tiempo real	Diseño centrado en el usuario; sin evaluación formal en el resumen
Lakshmipathy y Kumar [6]. Aplicación (India)	Ubicación, contactos de emergencia, recursos legales, defensa personal, comunidad	Entrevistas y encuestas previas; sin evaluación en el resumen
ALERTA MUJER. Diseño e implementación (Colombia)	Implementado, pendiente de verificación: botón, contacto, sacudida, widget, trayecto y protección automática; espera de 5 s con PIN o volumen; PIN de emergencia; aviso por correo y push con un reenvío a los 5 min; seguimiento público; grabación	553 pruebas automatizadas aprobadas el 27/10/2026; verificación funcional pendiente

B. Privacidad y riesgos de compartir la ubicación

Las aplicaciones de seguridad manejan justamente los datos que más exponen a una persona en riesgo. Ford et al. [4] encontraron que el 86,0 % de las aplicaciones revisadas pedía acceso a la ubicación al descargarse y el 61,6 % a los contactos, y en las reseñas aparecieron quejas por solicitudes de datos que las personas consideraban innecesarias. En la revisión de Sumra et al. [3], la privacidad figura como una preocupación central, y solo una parte de las aplicaciones ofrecía cifrado de datos o comunicación anónima.

El riesgo no se limita al abuso deliberado. Childs et al. [7] encuestaron a 896 personas que usan aplicaciones de ubicación compartida continua y entrevistaron a 23 que habían tenido experiencias incómodas. El malestar aparecía de tres formas que se van acumulando: límites personales que se traspasan, incomodidad que se prolonga y cambios de conducta en la vida diaria. Ma et al. [8] describen la dificultad de gestionar la privacidad de la ubicación en el teléfono, asociada a la fatiga y a configuraciones difíciles de usar; sus participantes pidieron procesos de consentimiento más detallados, que expliquen los riesgos de compartir la ubicación y no solo sus ventajas.

Estos hallazgos respaldan un criterio de ALERTA MUJER: la ubicación solo se comparte con los contactos de confianza durante una alerta y hasta dos horas después de que termina, mientras el enlace de seguimiento siga activo. El celular principal registra también la ubicación del día a día, que no se comparte con los contactos; la falta de un interruptor para desactivarla es un hallazgo abierto (S1). El tutorial explica la finalidad de los permisos, y el permiso de ubicación en segundo plano solo se pide al activar la protección automática. Ninguna de estas medidas ha sido evaluada con usuarias. En Colombia, el tratamiento de la ubicación, los contactos y el historial de alertas está sujeto a la Ley 1581 de 2012 [9], que establece principios como la finalidad, la libertad, el acceso y la circulación restringida, la seguridad y la confidencialidad. El sistema se diseña con esa norma como referencia, lo que no equivale a una verificación de cumplimiento.

C. Geolocalización en Android y restricciones técnicas

Android limita por diseño el seguimiento continuo. Desde Android 8.0 (API 26), una aplicación en segundo plano solo puede recibir actualizaciones de ubicación unas pocas veces por hora [10]. Si la aplicación apunta a Android 10 (API 29) o superior, el acceso a la ubicación en segundo plano exige además el permiso ACCESS_BACKGROUND_LOCATION [10]. Desde Android 14 (API 34), cada servicio en primer plano debe declarar su tipo, y un servicio de tipo location no puede crearse mientras la aplicación está en segundo plano, salvo que tenga concedido ese permiso [11].

La precisión y la batería imponen otros límites. Kim et al. [12] proponen ajustar el error permitido del GPS según el estado de la batería en Android y muestran, en un caso de navegación vehicular, que el ajuste adaptativo permitió terminar el trayecto, mientras que los métodos convencionales agotaron la batería antes de llegar. Thierry et al. [13] compararon datos de ubicación de teléfonos inteligentes con los de un GPS dedicado en 337 participantes: coincidían al agregarlos por día, pero se separaban en escalas menores a seis horas. Ninguno de los dos estudios trata situaciones de emergencia, aunque ambos muestran que la calidad de la ubicación depende de la configuración y del intervalo de análisis.

En ALERTA MUJER, la activación al agitar el teléfono, el trayecto y la protección automática están implementados con un módulo nativo en Kotlin y servicios en primer plano; la protección automática usa geocercas y el permiso ACCESS_BACKGROUND_LOCATION. Durante la alerta, el servidor reenvía el aviso una sola vez si ningún contacto lo abre en cinco minutos. Todo esto depende de permisos y de cómo maneja cada versión de Android los procesos en segundo plano, así que hay que probarlo en dispositivos físicos. Por esa razón, este trabajo habla de actualización periódica de la ubicación y no de seguimiento en tiempo real.

D. Arquitectura modular y evolución hacia microservicios

Los microservicios se han convertido en una referencia para sistemas que necesitan evolucionar por partes, pero su adopción tiene costos. Abgaz et al. [14] revisaron 35 estudios sobre la descomposición de aplicaciones monolíticas en microservicios y concluyeron que el área sigue en una etapa temprana: faltan herramientas, métodos que combinen datos estáticos, dinámicos y de evolución, y métricas y conjuntos de datos estandarizados para validar los resultados. Richardson [15] reúne los patrones con los que se suelen resolver problemas propios de esta arquitectura, como la comunicación entre servicios y la gestión de datos distribuidos.

En ese contexto aparece el monolito modular. Su y Li [16], mediante una revisión de literatura gris, lo describen como un enfoque que combina ventajas del monolito y de los microservicios; identifican tres frameworks y cuatro casos, y lo plantean como alternativa y también como paso intermedio antes de migrar. La evidencia empírica sobre la migración sigue siendo puntual. Barzotto y Farias [17] compararon las versiones monolítica y de microservicios de una funcionalidad de una institución financiera, y la segunda obtuvo mejores métricas de modularidad y menor uso de memoria y CPU; se trata de un único caso, publicado como preprint.

ALERTA MUJER siguió la ruta que describen Su y Li [16]. El backend empezó como un monolito modular y se convirtió en tres microservicios: Auth (identidad, dispositivos,

configuración y administración), Alert (alertas, notificaciones, zonas y recursos) y Evidence (evidencias). Sus nueve módulos siguen la arquitectura hexagonal, con un núcleo compartido (shared-kernel), llamadas internas por RPC y una base PostgreSQL con cinco esquemas; el monolito se retiró del repositorio el 25/10/2026, y pruebas de arquitectura con ArchUnit verifican que el dominio no dependa de los adaptadores. La descomposición no resuelve todos los problemas que señalan Abgaz et al. [14]: los servicios comparten la base de datos y el criterio para fijar sus límites no se ha documentado formalmente.

E. Autenticación y seguridad de aplicaciones

JSON Web Token (JWT), definido en el RFC 7519 [18], representa de forma compacta un conjunto de afirmaciones (claims) que pueden firmarse o cifrarse. Un token firmado protege la integridad de su contenido, pero no lo oculta, por lo que no conviene incluir en él datos sensibles. El RFC 8725 [19] reúne las prácticas recomendadas para su uso, como verificar el algoritmo, validar los claims y emplear algoritmos adecuados. Para evaluar la seguridad de una aplicación completa, OWASP mantiene dos estándares de verificación: ASVS [20], para aplicaciones web y servicios, y MASVS [21], para aplicaciones móviles. Ford et al. [4] muestran por qué esto importa en este dominio: sin acreditación ni pruebas independientes, una persona en riesgo no tiene forma de saber si la aplicación en la que confía es segura.

La autenticación de ALERTA MUJER se basa en tokens JWT con vigencia de 24 horas, firmados con un secreto que los servicios exigen para arrancar; las contraseñas se almacenan con BCrypt y el acceso se controla con dos roles, usuario y administrador. La aplicación móvil y la web añaden el token a las peticiones con un interceptor; el móvil lo renueva cuando el servidor responde 401, mientras que la web guarda el token en localStorage, un hallazgo abierto (S8). CORS está restringido al dominio de la web y las rutas de acceso limitan los intentos por minuto. La propuesta técnica toma OWASP ASVS como referencia, pero el sistema no ha pasado por una auditoría externa ni por pruebas de penetración. Por eso este trabajo no afirma el cumplimiento de esos estándares; su verificación queda como trabajo futuro.

En conjunto, los trabajos revisados coinciden en dos vacíos: las aplicaciones de seguridad personal rara vez se evalúan en condiciones reales y la privacidad de quienes las usan suele quedar en segundo plano. ALERTA MUJER todavía no resuelve ninguno de los dos. Su aporte en esta etapa consiste en reunir en una misma solución una aplicación móvil, un portal web y un panel administrativo, y en declarar con precisión qué está implementado, qué falta verificar y qué sigue pendiente.

III. METODOLOGÍA

A. Arquitectura del Sistema

Alerta Mujer utiliza un backend de microservicios para mantener separadas las funciones del sistema, de modo que cada parte tenga su propia responsabilidad. Hay dos interfaces: una plataforma web hecha con Angular y una app móvil hecha con React Native y Expo. Las dos dan acceso al sistema desde distintos dispositivos.

El backend está dividido en tres servicios: Auth (usuarias, cuentas, dispositivos, configuración y administración), Alert (alertas, avisos, seguimiento, trayectos, zonas y recursos) y Evidence (evidencias). Están hechos con Java y Spring Boot y reciben por HTTP las peticiones que llegan a la API desde las interfaces; no hay puerta de enlace, y cada cliente elige el servicio según la ruta.

En la web, Angular Router maneja la navegación entre páginas y las guardias de ruta controlan quién entra a cada sección. En el móvil, React Navigation organiza el paso entre pantallas, con varios tipos de navegación.

Como el frontend y el backend están separados, las interfaces le piden los datos a la API y no llevan dentro la lógica para gestionarlos. Eso también hace más fácil el mantenimiento y los cambios quedan más ordenados.

B. Selección de tecnologías

1) Decisión del framework backend

El backend está hecho con Java y Spring Boot, que encaja con la idea de microservicios: cada servicio se ocupa de lo suyo. La elección de Spring Boot se justifica por su integración con Spring Security, Spring Data JPA y herramientas de pruebas, además de permitir implementar una API REST en Java 17. Los tres servicios se despliegan por separado en Render, cada uno en su contenedor Docker.

Spring Boot se usa en los tres servicios: Auth, Alert y Evidence. Spring Security y JWT se encargan de la autenticación y de proteger las funciones que requieren sesión.

Los datos se guardan en PostgreSQL. Las interfaces hablan con el backend mediante endpoints de la API, para cosas como iniciar sesión, consultar los datos de la usuaria, gestionar alertas y administrar los contactos de emergencia.

2) Stack tecnológico

En la web se usa Angular 21.2.0 con TypeScript para las interfaces y sus componentes. Angular Router se encarga de la navegación, y las guardias de ruta separan lo que puede ver una usuaria de lo que puede ver un administrador. Los estilos van con Bootstrap 5 y Tailwind CSS.

En el móvil se usa React Native 0.81.5 con Expo 54. React Navigation maneja el paso entre pantallas y Axios hace las peticiones al backend. Las funciones del celular usan bibliotecas de Expo (ubicación, cámara y contactos), MapLibre para los mapas, Notifee y Firebase Cloud Messaging para las notificaciones, y un módulo nativo propio en Kotlin para la sacudida, la grabación de audio, el trayecto y la protección automática.

En el backend van Java y Spring Boot para los servicios, Spring Security y JWT para la autenticación y el control de acceso, PostgreSQL 16 como base de datos con migraciones Liquibase, Backblaze B2 para las evidencias y Resend para el correo. Los servicios se publican en Render y la web en Vercel.

El código se maneja con Git y GitHub, con las ramas develop, qa y main y cambios por pull request. GitHub Actions ejecuta las pruebas de los tres repositorios en cada cambio.

C. Módulos funcionales de Alerta Mujer

Alerta Mujer está dividida en módulos. Cada uno se encarga de una parte del sistema, y la web, la app móvil y el backend los usan juntos.

Los módulos se agrupan en tres servicios. Auth reúne identidad y cuentas (registro, inicio de sesión y recuperación de contraseña), dispositivos, configuración y administración. Alert reúne las alertas de emergencia, los avisos a los contactos y el seguimiento público, los trayectos y la protección automática, y las zonas y recursos de apoyo. Evidence guarda las grabaciones de audio y video.

Cada módulo tiene sus capas de dominio, aplicación y adaptadores, por eso se puede mantener por separado. Cuando un módulo necesita un dato de otro servicio, lo pide por un contrato interno y no lee sus tablas.

D. Patrones de diseño aplicados

1) Repository

El patrón Repository separa el acceso a los datos de la lógica del sistema, para que las consultas a la base de datos no se mezclen con todo lo demás. Así el código queda más ordenado, se pueden reutilizar consultas y cambiar esa parte sin tocar la lógica de negocio.

En Alerta Mujer, cada módulo accede a sus propios datos: usuarios, alertas, zonas y recursos. La persistencia se organiza mediante entidades y repositorios Spring Data JPA para cuentas, alertas y demás registros del dominio. En la arquitectura hexagonal los repositorios son puertos de salida, como AlertRepository y TripRepository, que implementan adaptadores de persistencia.

2) DTO (Data Transfer Object)

Un DTO define qué información viaja entre los componentes. Sirve para no mostrar los datos internos tal cual y para controlar qué entra y qué sale por la API.

En el proyecto se usa en la comunicación entre el backend y las interfaces web y móvil, por ejemplo al iniciar sesión, consultar usuarios o gestionar alertas. La API intercambia objetos de solicitud y respuesta para autenticación, perfiles y alertas. Por ejemplo, LoginRequest y RegisterRequest llegan al servicio Auth y AuthResponse devuelve el token; los controladores nunca exponen las entidades de la base de datos.

3) Service Layer

Service Layer pone la lógica de negocio en una capa de servicios, aparte de los controladores y del acceso a datos. En Alerta Mujer los servicios del backend se dividen por función: cuentas, alertas, zonas y recursos, y administración y moderación. Separarlos así hace más fácil cambiar cosas y probarlas. Los controladores REST reciben solicitudes, los servicios ejecutan las reglas y los repositorios consultan PostgreSQL. Esta separación se aprecia especialmente en el procesamiento de alertas: los casos de uso, como el de expiración (AlertExpirationService), solo conocen puertos.

E. Sistema de métricas (KPIs)

Los KPIs sirven para medir cómo anda el sistema: rendimiento, calidad y facilidad de uso. En Alerta Mujer se podrían medir el tiempo de respuesta de la API, los errores en las operaciones, la disponibilidad de los servicios, la cobertura de pruebas y lo que opinen las usuarias en las evaluaciones. Con las alertas, se puede contar cuántas se registran en un periodo y si crearlas y consultarlas funciona bien.

Cada indicador necesita una forma de medirse y un valor de referencia. Los números tienen que salir de pruebas y registros reales del proyecto, y una meta propuesta no se puede presentar como si fuera un resultado. El único resultado medido son las 553 pruebas automatizadas aprobadas el 27/10/2026 (351 del backend, 39 de la web y 163 del móvil), que GitHub Actions repite en cada cambio. No hay mediciones acreditadas de latencia, disponibilidad ni cobertura de código.

F. Buenas prácticas aplicadas

1) Clean Code

Clean Code son prácticas para que el código sea claro y fácil de mantener: nombres descriptivos, no repetir código y que cada función o componente haga una sola cosa. En un proyecto con web, móvil y varios servicios, una estructura ordenada ayuda a entender cada parte y a cambiar algo sin que se rompa lo demás. En la aplicación móvil se reutilizan componentes de interfaz, contextos de estado y servicios Axios; en Angular se separan componentes, servicios y rutas. Se detectó una pantalla de mapa con lógica extensa, por lo que la aplicación de Clean Code es desigual.

2) Principios SOLID

SOLID reúne recomendaciones para que el software sea ordenado y fácil de cambiar: cada pieza con una sola responsabilidad, poder extender sin reescribir y depender de abstracciones cuando tenga sentido.

En Alerta Mujer se ve en la separación entre módulos: la gestión de usuarios va aparte de la de alertas. En el frontend, los componentes de presentación van separados de la lógica de negocio. La responsabilidad única se refleja parcialmente en la separación de módulos de identidad, alertas y zonas, y en la inyección de dependencias de Spring. Los casos de uso dependen de puertos (interfaces) y no de los adaptadores, lo que aplica la inversión de dependencias; no se verificó la aplicación integral de los otros principios.

3) Validaciones multinivel

Validar es comprobar que los datos cumplan lo necesario antes de procesarlos o guardarlos. En Alerta Mujer importa mucho, porque se manejan datos de usuarias, contactos de emergencia y alertas.

Se valida en tres niveles. En el frontend, con los formularios. En el backend, con lo que llega a la API y las reglas de negocio. Y en la base de datos, para cuidar la integridad de lo guardado. Así la validación no depende solo de la pantalla. Los

formularios validan los campos; Spring Security y la lógica del servidor comprueban credenciales y roles; PostgreSQL impone restricciones de persistencia. No se dispone de una matriz de validaciones por campo ni de pruebas negativas exhaustivas.

G. Seguridad implementada

El sistema guarda datos personales, alertas y ubicación de las usuarias, así que hay que controlar el acceso y proteger lo que viaja entre las interfaces y el backend.

Según la documentación técnica del proyecto, el backend usa Spring Security y JWT para autenticar y controlar el acceso a las funciones protegidas. También hay que validar lo que llega a la API, manejar permisos según el tipo de usuario y evitar que alguien sin autorización vea o cambie información sensible. Lo mismo vale para las comunicaciones y los datos de ubicación.

IV. Implementación

A. Estructura del proyecto backend

El backend de Alerta Mujer está organizado en módulos, uno por cada responsabilidad del sistema. Así es más fácil desarrollar, mantener y conectar las partes entre sí.

Los servicios son tres: Auth, Alert y Evidence. Cada uno es una aplicación Spring Boot con su propio contenedor, y los tres usan un núcleo compartido para la seguridad, los errores y los contratos internos.

1) Módulo Identity (Identidad y Cuentas)

Aquí se manejan las usuarias y sus cuentas: la autenticación y la administración de sus datos. Para controlar el acceso a las funciones protegidas se usan Spring Security y JWT. Es la puerta de entrada a la plataforma: incluye la recuperación de contraseña con un código de seis dígitos válido 15 minutos y la eliminación de la cuenta por la usuaria.

2) Módulo Alert (Gestión de Alertas)

Este módulo se ocupa de las alertas de emergencia. Recibe las solicitudes que salen de las interfaces y permite crear y consultar alertas. Las alertas son una de las funciones principales del proyecto, y por eso este servicio se conecta con la información de las usuarias. La operación POST /api/alerts crea una alerta; el módulo también avisa a los contactos, mantiene el chat de la alerta y el seguimiento público, y administra los trayectos y la protección automática.

3) Módulo Resource (Zonas y Recursos)

Organiza las zonas y los recursos de apoyo, y deja que las interfaces consulten esos datos por la API. Le sirve a la usuaria que quiere ver información de su entorno. Los catálogos de zonas y recursos se cargan desde el panel administrativo.

4) Módulo Admin (Administración y Moderación)

Reúne la administración y la moderación del sistema, para los perfiles autorizados. Está separado para que las tareas administrativas no se mezclen con las funciones de las usuarias, y para entrar hay que pasar por la autenticación y la autorización del sistema. La administración solo ve el recorrido de las alertas activas o atendidas y, por cada contacto, el estado del envío y el parentesco, sin sus datos personales.

B. Estructura del proyecto frontend

Alerta Mujer tiene dos interfaces: una web en Angular y una app móvil en React Native con Expo. Las dos dan acceso a las funciones del sistema, pero cada una tiene su propia estructura y sus propias herramientas.

1) Frontend web

La web usa Angular 21.2.0 con TypeScript y se organiza en componentes, servicios, rutas y módulos funcionales.

Hay tres layouts (público, de usuarias y administrativo) que acomodan la presentación y la navegación según la sección. Angular Router maneja las rutas, y las guardias de ruta controlan el acceso según el tipo de usuario y si inició sesión.

Los módulos funcionales son autenticación, panel de usuarias, ubicación, historial de alertas, alertas de mis contactos, evidencias, contactos de emergencia y configuración, más las secciones de administración y moderación. La interfaz está traducida al inglés, al portugués y al francés. Los estilos van con Bootstrap 5 y Tailwind CSS, y los servicios de Angular hablan con el backend a través de la API.

2) Frontend móvil

La app usa React Native 0.81.5 y Expo 54, con TypeScript. React Navigation maneja el paso entre pantallas, con pestañas, menús laterales y pilas de pantallas. Axios hace las peticiones HTTP al backend, y hay contextos para los estados compartidos: autenticación, contactos, evidencias, notificaciones, tema y localización.

Para las funciones del celular están las bibliotecas de Expo y React Native de ubicación, cámara, notificaciones y contactos. En solicitudes protegidas se utiliza un token JWT de acceso enviado mediante la cabecera Authorization con el esquema Bearer. Cuando el servidor responde 401, el cliente renueva el token y repite la petición.

C. Flujo de una petición

Todo empieza cuando una usuaria hace algo en la web o en la app: iniciar sesión, consultar datos, crear una alerta.

La interfaz manda una petición HTTP al backend desde sus servicios. Si la operación necesita autenticación, la petición lleva el token de acceso. En el flujo de alertas, el cliente solicita POST /api/alerts, el controlador delega el procesamiento al servicio y la persistencia se realiza mediante repositorios. La creación SOS sin conexión no está encolada, por lo que una falla de red puede impedir el registro.

El controlador HTTP delega en el servicio de aplicación correspondiente. Ese servicio valida los datos, ejecuta la lógica y, si hace falta, consulta la base de datos. Al final el backend responde por la API y la interfaz le muestra a la usuaria el resultado o el mensaje.

Así la presentación, el procesamiento y los datos quedan separados. Las pruebas del backend utilizan JUnit, Mockito y ArchUnit: 351 aprobaron el 27/10/2026 (shared-kernel 16, Auth 114, Alert 196 y Evidence 25).

V. Pruebas y validación

A. Estrategia de pruebas

Para saber si Alerta Mujer funciona bien hay que probar sus tres partes: el backend, la plataforma web y la app móvil. Las pruebas confirman que cada función responde como debe y que las partes se comunican entre sí.

Se revisan los servicios de autenticación y cuentas, alertas, zonas y recursos, y administración y moderación. También se prueba la comunicación entre las interfaces y el backend, y el comportamiento de los contactos de emergencia y la ubicación.

Además hay que mirar los controles de acceso, la validación de datos y el manejo de errores en las peticiones a la API. Pesan más en este proyecto, porque el sistema guarda datos personales y datos de situaciones de emergencia.

B. Pruebas del backend (Spring Boot)

1) Pruebas unitarias

Las pruebas unitarias revisan una pieza del backend a la vez, aislada del resto, para confirmar que hace bien lo que le toca.

En Alerta Mujer se pueden aplicar a la autenticación y la gestión de usuarios, a las alertas y a la administración de zonas y recursos. También sirven para ver cómo reaccionan los servicios cuando reciben datos incorrectos o algo falla a mitad de una operación. No se dispone de un informe reproducible de una campaña Postman ni de una matriz completa de respuestas HTTP. Las pruebas manuales de endpoints quedan pendientes de documentar con URL, método, datos anonimizados, código y fecha.

2) Pruebas de API

Estas pruebas revisan la comunicación entre las interfaces y los servicios. Cada endpoint debe recibir lo que espera, procesar los datos y devolver una respuesta correcta.

Se pueden probar el registro y el inicio de sesión, la consulta y actualización de los datos de la usuaria, la creación y consulta de alertas, los contactos de emergencia y la consulta de zonas y recursos. También hay que ver qué pasa en los endpoints protegidos cuando la petición llega sin autenticación o con credenciales inválidas. La compilación web de producción fue satisfactoria. El 27/10/2026 pasaron las 39 pruebas de la web. Aun así, no hay pruebas de extremo a extremo de las pantallas, así que no se puede dar por validado todo el portal.

C. Pruebas del frontend web (Angular y TypeScript)

En la web se prueba cómo se comportan los componentes, los formularios, la navegación y los servicios que hablan con el backend.

1) Pruebas de componentes y servicios

Las pruebas de componentes comprueban que la pantalla muestre bien la información y responda a lo que hace la usuaria. Las de servicios revisan el envío de peticiones a la API y cómo se procesan las respuestas.

En Alerta Mujer se pueden probar la autenticación, el panel de usuarias, el historial de alertas, los contactos de emergencia y las secciones administrativas, además de las guardias de ruta según el perfil y si hay sesión iniciada. No hay pruebas automatizadas de las guardias de ruta ni de los interceptores.

D. Pruebas del frontend móvil (React Native y Expo)

En la app se prueban la navegación entre pantallas, los formularios y la comunicación con el backend.

Importa sobre todo la autenticación, el acceso a las funciones principales y todo lo de alertas y contactos de emergencia. Las funciones que dependen del celular (ubicación, cámara, notificaciones) se prueban si ya están implementadas, tanto con los permisos concedidos como sin ellos.

Con esto se detectan fallas de navegación, errores en las peticiones HTTP y problemas con los permisos del dispositivo. Flujo de autenticación: implementación presente, prueba E2E documentada pendiente. Creación de SOS: implementación presente, verificación integral pendiente. Aviso automático a contactos: implementado y probado de forma manual entre dos teléfonos el 23/10/2026, sin evidencia archivada. El APK v1.0.0 se instaló en un Samsung SM-A125M y en un Infinix X678B; no se documentó una matriz con más dispositivos. Consulta de zonas: implementación parcial y datos de prueba limitados. Fecha de ejecución de estos flujos: no registrada.

E. Pruebas de integración

1) Flujos funcionales

Las pruebas de integración confirman que varias partes del sistema funcionan juntas. El primer flujo a probar es el de autenticación: la interfaz manda las credenciales al backend, recibe la respuesta y deja entrar o no según el resultado.

El segundo es el de las alertas de emergencia: se crea la alerta desde la interfaz, la solicitud llega al backend, se procesa y la app recibe la respuesta. También se prueban los contactos de emergencia y la consulta de zonas y recursos. En la ubicación en tiempo real se revisa cómo se comporta la app y cómo viajan los datos, según los requisitos del proyecto.

Estos flujos son casos de prueba propuestos. Para llamarlos “validados” hay que ejecutarlos y anotar qué pasó. La auditoría identificó riesgos de CORS permisivo y de acceso amplio de la administración a las ubicaciones, que ya se corrigieron; siguen abiertos el almacenamiento del JWT en la web y los registros de consola del móvil. No se dispone de un informe de pentesting ni de casos de seguridad ejecutados con resultados verificables.

F. Pruebas de seguridad

Las pruebas de seguridad buscan problemas en el acceso a la información, la autenticación, los permisos y el manejo de los datos que entran al sistema.

En Alerta Mujer se miran la autenticación con Spring Security y JWT, los endpoints protegidos y la separación de permisos entre usuarias y administradores. También hay que comprobar la validación de los datos de entrada y que nadie sin autorización pueda ver datos personales ni ubicaciones. Como referencia para los riesgos en aplicaciones web y móviles se puede usar OWASP.

Cada prueba se documenta con el caso evaluado, el problema encontrado y lo que se hizo para arreglarlo. Sin esa

evidencia no se puede decir que el sistema pasó una evaluación de seguridad completa. No se encontraron actas con participantes, tareas observadas ni resultados de una prueba formal de usabilidad. La validación con usuarias queda pendiente.

G. Validación con usuarios

La validación con usuarios muestra si las funciones se entienden y si las interfaces hacen fáciles las tareas.

Se les puede pedir que inicien sesión, exploren las opciones, administren sus contactos de emergencia y usen lo de alertas, y se observa qué tan fácil les resulta moverse por la web y por la app. Si se hace, hay que anotar cuántas personas participaron, qué tareas hicieron, cómo se hizo la prueba y qué resultó. Si todavía no se ha hecho, se presenta como actividad pendiente y no como una validación terminada. El 27/10/2026 pasaron 163 pruebas del móvil (Jest) con TypeScript sin errores, 39 de la web (Vitest) y 351 del backend (JUnit 5, Mockito y ArchUnit). Cobertura porcentual: no medida.

H. Métricas de calidad de código

Las métricas muestran el estado técnico del proyecto: errores detectados, cobertura de pruebas y cumplimiento de las reglas de análisis estático. Se pueden medir por separado en el backend (Java y Spring Boot), la web (Angular y TypeScript) y la app móvil (React Native y Expo).

Lo que se puede documentar es cuántas pruebas se ejecutaron y cuántas pasaron, la cobertura de código si hay informe, y los problemas que encuentren las herramientas de análisis estático. Los números salen de los informes del desarrollo. Si todavía no existen, se dejan como parte del plan de evaluación. KPI de pruebas al 27/10/2026: móvil 163, web 39 y backend 351, todas aprobadas. Latencia y disponibilidad: sin medición.

VI. Resultados

A. Indicadores clave de rendimiento (KPIs)

Los KPIs muestran cómo va Alerta Mujer y qué tanto avanzaron sus funciones principales. Como el proyecto tiene app móvil, plataforma web y un backend modular, hay que medir las interfaces y también cómo se comunican los servicios entre sí.

Los indicadores que se pueden usar son las pruebas funcionales aprobadas, el porcentaje de solicitudes procesadas bien, el tiempo de respuesta de la API y la disponibilidad de los servicios. También se puede ver el uso de las funciones principales, como la gestión de contactos de emergencia y la activación de alertas.

Cada indicador necesita un método de medición y una fuente. Los valores salen de las pruebas ejecutadas o de los datos del sistema, y que una función esté programada no quiere decir que funcione. No se ejecutó una campaña documentada de carga con número de solicitudes, concurrencia y percentiles de latencia. El rendimiento permanece pendiente de medición.

B. Métricas de rendimiento

Estas métricas muestran cómo responde el sistema cuando le llegan solicitudes. Interesa sobre todo el backend y los módulos de identidad y cuentas, alertas, zonas y recursos, y administración y moderación.

La principal es el tiempo de respuesta de la API: cuánto tarda el backend en procesar una solicitud y contestar. También se mide la proporción de solicitudes exitosas y fallidas, y cómo se portan los servicios cuando llegan varias a la vez. Con eso se ven los retrasos y los componentes que hay que mejorar.

Para poner números hay que correr pruebas de rendimiento y anotar en qué condiciones se hicieron. Desde el 28/10/2026 un monitor de UptimeRobot consulta cada cinco minutos el estado de los tres servicios, lo que además evita que el alojamiento gratuito los suspenda; todavía no hay una ventana de observación suficiente para calcular un porcentaje de disponibilidad.

C. Disponibilidad del sistema

La disponibilidad es el tiempo que los componentes están funcionando y pueden atender solicitudes. En Alerta Mujer cuenta mucho, porque algunas funciones sirven para pedir ayuda y manejar alertas de emergencia.

Se puede evaluar en los tres microservicios (autenticación, alertas y evidencias), la conexión con la base de datos y la comunicación entre el backend y las interfaces. También hay que ver qué pasa cuando un servicio no responde o se pierde la conexión con el servidor.

Para sacar un porcentaje se necesita registrar los periodos de funcionamiento y las caídas durante un tiempo definido. Sin esas mediciones no se puede dar un porcentaje real. No existen registros analíticos verificables de usuarias activas o frecuencia de uso. La adopción del sistema está pendiente de evaluación.

D. Adopción de usuarios

La adopción dice cuánta gente usa Alerta Mujer de verdad. No es lo mismo que el total de cuentas: una cuenta creada no siempre es alguien que usa el sistema seguido.

Se pueden mirar las cuentas registradas, las usuarias activas en un periodo y la frecuencia con que usan las funciones principales. También cuántas personas terminan procesos como el registro, el inicio de sesión y la configuración de sus contactos de emergencia.

Para interpretar esos datos hacen falta registros confiables y una definición clara de cada métrica. Si todavía no se han hecho pruebas con usuarias o no hay suficiente información de uso, la adopción queda pendiente de evaluar. No se aplicó una encuesta documentada de satisfacción con muestra y cuestionario identificables. No se atribuye calificación de satisfacción al producto.

E. Satisfacción del usuario

La satisfacción muestra qué piensan las personas sobre lo fácil que es usar el sistema, cómo están organizadas las pantallas y qué tan útiles son las funciones.

Se puede aplicar una encuesta después de que prueben la web o la app, con preguntas sobre moverse entre pantallas, entender las opciones, gestionar contactos de emergencia y encontrar lo relacionado con las alertas. Los resultados se analizan con las respuestas reales y se anota cuántas personas participaron, qué se preguntó y cómo se calculó el resultado.

Si no se ha hecho la encuesta, no se le puede poner una calificación de satisfacción al proyecto. Liquibase gestiona las migraciones del esquema PostgreSQL. No se aportaron resultados medidos de integridad referencial, latencia de consultas o carga; esas pruebas quedan pendientes.

F. Métricas técnicas del sistema

Las métricas técnicas revisan cómo están funcionando, y qué tan mantenibles son, la base de datos, la API, el backend y las interfaces web y móvil. Entre ellas: pruebas aprobadas, errores detectados, tiempos de respuesta y problemas de integración. Sirven para ver qué funciona bien y qué hay que ajustar antes de dar el sistema por listo.

1) Base de datos PostgreSQL

PostgreSQL guarda la información del sistema. Hay que comprobar que consultar, crear y actualizar datos funcione como necesitan los servicios.

Se puede revisar la integridad de los registros, las relaciones entre entidades, qué hace la base con datos inválidos, el tiempo de las consultas y si los servicios logran conectarse. Los resultados se documentan con las pruebas hechas y los datos obtenidos, y no se le atribuye a PostgreSQL ningún rendimiento sin evidencia. POST /api/alerts: ruta identificada, sin registro de código HTTP de una campaña API reproducible. Autenticación y consultas: implementadas, sin tabla de códigos verificados. Respuestas 401/403 y errores de validación: pendientes de pruebas documentadas.

2) API REST y backend

El backend separa las responsabilidades en servicios, y eso permite probar cada uno por separado y después comprobar cómo se comunican entre sí, sin una puerta de enlace.

En la API REST se revisan la disponibilidad de los endpoints, la validación de las solicitudes, el formato de las respuestas y el manejo de errores, además de la autenticación y la autorización en las operaciones que piden permisos. Para documentarlo se anotan las solicitudes ejecutadas, los códigos de respuesta, los errores y el resultado de cada caso. Con eso se sabe qué está verificado y qué necesita ajustes. Las estadísticas estáticas, la recuperación de contraseña simulada y las integraciones administrativas incompletas que registraron las primeras revisiones ya se corrigieron en el código; sigue pendiente la accesibilidad del móvil.

3) Frontend

En la web se revisan la navegación, los formularios, la protección de rutas y cómo se muestran los datos que llegan de la API. En la app móvil, además, las interacciones propias del celular y los permisos que piden algunas funciones.

También hay que ver cómo responden las interfaces cuando una solicitud falla o el servidor no está disponible. Los resultados dejan los problemas encontrados y lo que se corrigió, separando lo que ya se probó de lo que sigue pendiente. Objetivo de integrar clientes y backend: parcial, existe código y compilación. Objetivo de registrar SOS: implementado, sin E2E documentado. Aviso a contactos: implementado (correo y push), probado de forma manual entre dos teléfonos, sin evidencia archivada. Gestión de zonas y administración: implementada. Pruebas automatizadas: 553 aprobadas el 27/10/2026.

G. Comparación con los objetivos establecidos

Esta comparación muestra cuánto avanzó Alerta Mujer: qué está desarrollado, qué está probado y qué falta.

Se revisan los objetivos de la app móvil y la plataforma web, las alertas de emergencia, los contactos de confianza, la ubicación, los microservicios y las funciones administrativas. Lo ideal es poner al lado de cada objetivo su evidencia. Que un módulo exista se comprueba con el código y la interfaz, pero que funcione bien solo lo demuestran las pruebas.

Así queda claro en qué estado está el proyecto y qué funciones necesitan más validación para seguir mejorándolo. En el código, los tres servicios se despliegan por separado y sus nueve módulos siguen la estructura hexagonal (dominio, aplicación con puertos y adaptadores), comprobada con pruebas ArchUnit.

VII. Discusión

A. Impacto de la arquitectura hexagonal y las arquitecturas del frontend

Cada parte de Alerta Mujer usa la arquitectura que mejor le queda. El backend utiliza microservicios con arquitectura hexagonal. La app móvil combina organización por funcionalidades (Feature-Based) con el patrón MVVM. La web también se organiza por funcionalidades.

1) Arquitectura hexagonal en el backend

La arquitectura hexagonal separa la lógica principal del sistema de las cosas externas con las que tiene que hablar: la base de datos, los controladores HTTP u otros canales. La idea es que la lógica de negocio no dependa de una tecnología concreta.

En Alerta Mujer ayuda a repartir las responsabilidades de los tres microservicios (Auth, Alert y Evidence) y hace más fácil entender qué hace cada módulo.

Con esta separación se puede probar la lógica de negocio sin levantar todo lo externo, y el mantenimiento se simplifica. Pero eso solo pasa si las dependencias están bien ordenadas y la arquitectura se aplica igual en todos los servicios.

2) Feature-Based y MVVM en la app móvil

La app agrupa el código por funcionalidad: las pantallas, los componentes y los servicios de una misma función viven juntos.

Además usa MVVM (*Model-View-ViewModel*). La vista muestra la información y recibe lo que hace la usuaria, el ViewModel maneja el estado y la lógica de presentación, y el modelo representa los datos y sus operaciones.

Juntos organizan el código por funciones y por responsabilidades, y la lógica no se amontona en un solo componente. Se nota en autenticación, alertas, contactos de emergencia y ubicación, que mantienen cada una lo suyo. No todas las funciones tienen ViewModel, por lo que MVVM es parcial. La revisión encontró que la separación por funcionalidades facilita la navegación del proyecto, pero también una concentración de lógica en la pantalla de mapa Es una oportunidad concreta de refactorización.

3) Feature-Based en la plataforma web

La web agrupa el código por funcionalidad en vez de meter todos los componentes en una sola estructura. Autenticación, panel de usuarias, ubicación, alertas y administración quedan separadas.

Así es fácil encontrar los archivos de cada función y se puede trabajar en una sección sin tocar las demás. Los componentes, servicios y rutas de cada módulo quedan juntos. Funciona bien para una plataforma con varios perfiles y áreas de trabajo, siempre que las responsabilidades estén bien delimitadas y se siga el mismo criterio en todo el código.

4) Análisis general de las decisiones arquitectónicas

Cada decisión responde a lo que necesita su parte. La hexagonal separa la lógica de negocio de lo externo en el backend, Feature-Based y MVVM ordenan las funciones y la presentación en la app móvil, y Feature-Based organiza la web por áreas.

Dan una base para mantener y ampliar el sistema, pero la calidad no depende solo del patrón elegido. Cuenta cómo se aplica en el código y qué tan claras quedan las responsabilidades. Por eso la evaluación tiene que mirar la estructura interna de cada aplicación y también cómo se comunican el backend, la web y la app. Pruebas automatizadas al 27/10/2026: móvil 163, web 39 y backend 351. No se cuenta con valores acreditados de latencia, disponibilidad, satisfacción o entrega de avisos a contactos.

B. Análisis de KPIs

Los KPIs sirven para revisar si Alerta Mujer cumple los requisitos funcionales y no funcionales de su SRS (Especificación de Requisitos de Software). Como el sistema junta una app móvil, una plataforma web y un backend con API REST, hay que mirar las funciones y también cómo se comunican los componentes entre sí.

Según el alcance del SRS, lo primero que se debe evaluar es la activación de alertas de emergencia. El sistema prevé varios métodos de activación, el aviso a los contactos por los dos canales planificados (notificación push en la aplicación mediante FCM y correo electrónico) y el seguimiento de la ubicación mientras la alerta sigue activa. De ahí salen tres indicadores: el porcentaje de pruebas de activación aprobadas, el tiempo entre que se pide la alerta y se procesa, y el resultado de las pruebas de entrega en cada canal, una vez estén implementados.

Después vienen los contactos de emergencia: porcentaje de pruebas aprobadas en registro y consulta, y que la información se maneje según los permisos. La ubicación, el historial de alertas, las evidencias y la consulta de recursos de ayuda se evalúan igual, con casos de prueba ligados a sus requisitos.

En lo técnico se mide el tiempo de respuesta de la API REST, las solicitudes procesadas correctamente y los errores de las pruebas de integración, que muestran dónde falla la comunicación entre la app, la web y el backend.

El SRS fija algunas metas, como despachar la alerta hacia el correo o FCM en máximo 2 segundos (RNF-REN-001), pero todavía no se han medido. Hay además un problema conocido: en el plan gratuito de Render cada servicio se suspende a los 15 minutos sin uso y tarda hasta unos 90 segundos en volver a arrancar, lo que afecta directamente ese tiempo; el equipo lo mitiga con un monitor que mantiene despiertos los servicios. Son criterios de evaluación, y sus resultados se documentan cuando se ejecuten las pruebas.

C. Patrones de diseño: evaluación

El SRS planteó cinco patrones: Observer, Strategy, Singleton, Chain of Responsibility y Adapter. En el código los cinco están, aunque varios terminaron en un lugar distinto al que preveía el SRS, y además se usan otros. Aquí se explica cada uno según cómo quedó implementado.

1) Observer

Aparece en el backend y en la app. En el backend, cuando una alerta se activa, se termina o entra en advertencia, se publica un evento de Spring (por ejemplo, AlertWarningEvent) y el módulo de notificaciones lo escucha para avisar a los contactos. Así el módulo de alertas no necesita saber cómo se mandan los avisos. En la app, el módulo nativo emite eventos como onShake y onVolumeCancel, y JavaScript solo se entera cuando el sensor detecta algo.

2) Strategy

El SRS lo pensaba para los métodos de activación, pero en el código quedó en el envío de correos. ResendEmailSender y BrevoEmailSender cumplen la misma interfaz, y cuál se usa depende de la configuración. Para pasar de un proveedor a otro no hay que tocar la lógica de las alertas.

3) Singleton

No hay una clase escrita a mano como Singleton. Lo cumplen los beans de Spring: cada servicio se crea una sola vez y esa instancia la comparte todo el microservicio. El patrón está, solo que lo maneja el framework.

4) Chain of Responsibility

El SRS lo proponía para pasar de un canal de aviso a otro si el primero fallaba. En el código quedó en la seguridad: cada petición pasa por una cadena de filtros (JwtAuthenticationFilter, InternalKeyFilter y el límite de intentos) antes de llegar al controlador, y si un filtro la rechaza, ahí se corta. Los avisos a los contactos van por dos canales: la

notificación push dentro de la aplicación ALERTA MUJER, mediante Firebase Cloud Messaging (FCM), y el correo electrónico. SMS y WhatsApp se descartaron. Como la notificación push solo llega a quien tiene la aplicación instalada, el contacto de confianza debe tenerla. Ambos canales están implementados y se probaron de forma manual entre dos teléfonos, sin evidencia archivada. Falta probar qué pasa cuando un canal responde bien, cuando un proveedor falla y cuando el proceso pasa al siguiente. También hay que separar el intento de envío de la confirmación de que el mensaje llegó.

5) Adapter

Todo lo que está en adapter/out es un adaptador: persistencia, correo, Firebase y almacenamiento. Cada uno traduce una tecnología externa a la interfaz que espera el dominio, así que cambiar de proveedor no obliga a tocar la lógica principal. Falta probar cómo manejan las respuestas y los errores de cada servicio.

Además de esos cinco, el backend usa otros. Los puertos como AlertRepository y TripRepository siguen el patrón Repository: el dominio guarda y pide datos sin saber que detrás hay PostgreSQL. Entre microservicios hay un Proxy: RpcClientFactory crea con Proxy.newProxyInstance un cliente con el que un servicio llama a otro como si fuera un objeto local, y por debajo la llamada se vuelve una petición HTTP interna. Los objetos del dominio se crean con métodos de fábrica como SafeTrip.created(...), SafeTrip.automatic(...), ProtectionDevice.enroll(...) y Device.created(...), que ya aplican las reglas. Los controladores reciben y devuelven DTO, así que la API nunca expone las entidades de la base de datos. Spring inyecta las dependencias por constructor, y en las pruebas los puertos se cambian por mocks. Por último, siete tareas con @Scheduled revisan alertas vencidas, señal perdida, trayectos y retención de evidencias, de modo que el servidor actúa aunque el celular se quede sin señal.

En la app móvil, ThemeContext, AuthContext, LocaleContext y ContactsContext usan Provider/Context para que el tema, la sesión, el idioma y los contactos estén disponibles en cualquier pantalla con un hook. Todas las peticiones pasan por un interceptor de Axios que agrega el JWT y renueva la sesión cuando el servidor responde 401. Si no hay internet, las ubicaciones y las evidencias quedan en una cola y se suben cuando vuelve la conexión; la creación de la alerta permite reintentar, pero sin conexión no puede registrarse. El módulo Kotlin shake-sensor hace de puente (Bridge) entre JavaScript y funciones que Android solo deja usar desde código nativo.

Con esto se sabe dónde está cada patrón en el código. Falta comprobar con pruebas que se comporten bien en una emergencia real.

D. Sistema de métricas

Las métricas deben mostrar si se cumplen los requisitos, si las funciones principales andan y qué tan buena es la calidad técnica. Se agrupan en cuatro.

Las funcionales cuentan los casos de prueba ejecutados y aprobados en registro e inicio de sesión, contactos de emergencia, activación de alertas, historial y acceso a zonas y recursos.

Las de comunicación y rendimiento miden el tiempo de respuesta de los endpoints, la proporción de solicitudes exitosas y fallidas, y los resultados de las pruebas de integración entre clientes y backend. En las notificaciones se distingue entre lo que acepta el proveedor y las entregas que se pueden confirmar.

Las de seguridad y privacidad parten de que el SRS prevé Spring Security y JWT, permisos y comunicaciones seguras. Se prueba el acceso a los endpoints protegidos, la separación de permisos y el manejo de los datos personales y de ubicación.

Las de calidad técnica recogen los resultados de las pruebas automatizadas, la cobertura de código si hay informe y los errores del desarrollo.

Cada métrica necesita una definición, un método de cálculo y una fuente. El SRS fija requisitos y criterios de diseño, pero no tiene datos para dar porcentajes, tiempos o puntajes. Esos salen de pruebas reales.

E. Lecciones aprendidas

Cada componente se organiza según lo que tiene que hacer. Backend, app y web son un mismo sistema, pero cada uno usa un enfoque distinto.

En el backend, la arquitectura hexagonal adaptada separa dominio, aplicación e infraestructura, para que las reglas de negocio no dependan de la comunicación ni del acceso a datos. Para sostener esa separación hay que definir bien los puertos, los adaptadores y las dependencias.

En la app móvil, Feature-Based agrupa lo de cada funcionalidad y MVVM separa vista, ViewModel y modelo. El SRS suma Context API para el estado global. Todo busca que la app quede ordenada y que los componentes se actualicen cuando cambia algo compartido.

En la web, Feature-Based separa autenticación, portal de la usuaria y administración. El SRS describe core/ para lo transversal, features/ para las funcionalidades y shared/ para los componentes reutilizables, con Angular Router, guards e interceptores para la navegación, el acceso y las peticiones HTTP.

La integración se piensa desde el diseño. Alertas, ubicación, contactos y evidencias necesitan que interfaces, API y servicios intercambien datos según contratos definidos. Si los datos enviados, los endpoints o las respuestas no coinciden, aparecen errores que afectan todo el sistema.

Las dependencias externas también pesan: mensajería, ubicación e internet. El diseño y las pruebas tienen que contar con caídas de conexión, permisos no concedidos y proveedores de notificaciones que no responden.

Y en un sistema pensado para situaciones de riesgo no alcanza con que las pantallas se vean bien. Hay que verificar flujos completos, permisos, errores y manejo de datos, y no dar por confiable una función solo porque aparece en la interfaz.

Completar pantallas antes de cerrar los contratos de la API produjo discrepancias en las funciones administrativas, y una compilación exitosa no reemplaza las pruebas de extremo a extremo del SOS. Los trabajos de Sumra et al. [3], Ford et al. [4] y el prototipo SAM [5] ofrecen comparaciones pertinentes: revisiones de apps y problemas de confiabilidad, privacidad y diseño centrado en usuarias. ALERTA MUJER aún no cuenta con validación empírica que permita afirmar superioridad.

F. Relación con trabajos previos

Alerta Mujer se parece a otras soluciones para pedir ayuda, avisar a redes de apoyo y consultar recursos de atención en situaciones de riesgo. El SRS plantea botón de pánico, ubicación en tiempo real, contactos de emergencia, historial y directorio de instituciones de apoyo.

En lo técnico se parece a los sistemas que separan el backend de los clientes, y patrones como Adapter, Repository, Observer o Strategy son comunes en sistemas que integran servicios externos.

Pero para comparar con otros sistemas hay que consultar fuentes académicas y documentación de proyectos similares: funciones, tecnologías, seguridad y limitaciones. El SRS muestra los objetivos del proyecto, pero no sirve para decir que Alerta Mujer funciona mejor que otras soluciones. El 27/10/2026 pasaron 553 pruebas automatizadas (163 del móvil, 39 de la web y 351 del backend). El flujo completo de notificación SOS y el rendimiento no han sido validados.

VIII. Conclusiones

A. Resumen de hallazgos

El SRS define una solución, con app móvil y plataforma web, para apoyar a las mujeres ante situaciones de riesgo. Cubre autenticación, activación de alertas, contactos de emergencia, ubicación, historial, evidencias, zonas y recursos de ayuda, y herramientas administrativas.

Cada componente tiene su arquitectura: el backend es hexagonal adaptada (dominio, aplicación e infraestructura), la app usa MVVM organizado por funcionalidades y la web, Feature-Based. En el backend, Adapter y Repository aíslan el dominio, los eventos de Spring (Observer) avisan a los contactos, Strategy permite cambiar de proveedor de correo, Chain of Responsibility ordena los filtros de seguridad y un Proxy maneja las llamadas entre microservicios. En la app se usan Context para el estado global, un interceptor de Axios, colas con reintentos para cuando no hay señal y un puente nativo en Kotlin para los sensores.

Eso da una base para organizar el desarrollo, pero que funcione hay que comprobarlo con la implementación y las pruebas. Por ahora lo que se puede afirmar es el alcance, las decisiones de arquitectura y los criterios de diseño. Rendimiento, disponibilidad, seguridad real y satisfacción de las usuarias necesitan resultados de pruebas. Según el SRS v2.6, de los 103 requisitos funcionales 99 están implementados en código, 3 en desarrollo y 1 se retiró por decisión del equipo; de los 25 no funcionales, 11 están implementados, 7 en desarrollo y 7 siguen como propuesta. Ninguno está verificado. La evidencia de pruebas automatizadas respalda partes del código, no la entrega efectiva de avisos ni el uso en emergencias reales.

B. Contribuciones

Alerta Mujer junta una app móvil y una plataforma web para activar alertas de emergencia, compartir la ubicación con contactos de confianza, ver el historial y consultar recursos de ayuda.

Aporta una arquitectura que separa las responsabilidades del backend, la app y la web, y una forma de manejar eventos y notificaciones con patrones de diseño, que puede facilitar que el sistema crezca si se aplican bien. También define los requisitos para administrar contactos, consultar zonas seguras e inseguras, acceder a recursos de apoyo y gestionar información desde el panel administrativo.

Además, es una forma de aplicar lo aprendido en análisis, diseño y desarrollo a un problema social. Su aporte se valora según lo que esté implementado y verificado de verdad, separado de lo que sigue siendo requisito. El principal reto técnico observado fue integrar correctamente interfaces, servicios y condiciones de conectividad sin confundir una pantalla terminada con una emergencia atendida; por eso la prioridad es probar el flujo completo antes de presentar el sistema como confiable.

C. Limitaciones

El SRS identifica condiciones de las que depende el funcionamiento: conexión a internet, permisos del dispositivo (ubicación y otros) y disponibilidad de los servicios externos de notificaciones. Si un proveedor no está configurado o no responde, el envío por varios canales se afecta, y si el celular no deja usar la ubicación, no se puede compartir.

También falta comprobar la integración entre los clientes y el backend: que los endpoints, los datos y las respuestas coincidan con los contratos. Tener autenticación y autorización en el diseño no reemplaza las pruebas de seguridad, y la protección de los datos personales, los permisos y la ubicación tiene que evaluarse. De los hallazgos de seguridad de la auditoría ya se corrigieron, entre otros, el acceso amplio de la administración (S2), las credenciales en el historial (S3 y S16), la eliminación de la cuenta (S11) y el permiso de ubicación en segundo plano (S14); siguen abiertos la ubicación diaria sin interruptor (S1), el token de la web en localStorage (S8), los registros de consola (S9) y el texto «Grabación encriptada» (S18). Los textos de la política de privacidad y de los términos son provisionales.

Por último, no hay mediciones de disponibilidad, tiempos de respuesta, cobertura de pruebas ni satisfacción; el SRS fija algunas metas, pero no resultados. Sin medirlas no se pueden sacar conclusiones cuantitativas.

D. Trabajos futuros

Lo primero es completar la matriz de trazabilidad, que ya relaciona cada requisito con sus casos de uso, elementos de diseño y casos de prueba, con el resultado archivado de cada

prueba. Así se ve qué está implementado, qué está validado y qué falta.

Luego, completar las pruebas de integración entre la app, la web y el backend: registro e inicio de sesión, contactos, activación de alertas, historial y zonas y recursos. También conviene evaluar los métodos de activación del botón de pánico y qué hace el sistema cuando falla un canal de notificación, mirando los intentos de envío, las respuestas de los proveedores y el manejo de errores, sin suponer que una solicitud enviada es una alerta recibida. Con la ubicación y las evidencias, lo mismo: permisos, conectividad, protección de datos y qué pasa cuando falta algún recurso.

En lo técnico, medir el rendimiento de la API, documentar las pruebas automatizadas y registrar los errores, para priorizar mejoras. Y hacer pruebas de usabilidad con participantes, sobre todo en las operaciones que tienen que ser fáciles de usar en una situación de tensión o emergencia.

E. Reflexión final

Alerta Mujer es una propuesta para apoyar la seguridad de las mujeres con alertas, comunicación y acceso a recursos de ayuda: una app móvil y una plataforma web conectadas a un backend organizado por módulos de negocio.

El SRS da la base para organizar el sistema. La arquitectura de cada componente y los patrones definidos delimitan responsabilidades y responden a necesidades concretas: eventos, activación de alertas y servicios externos.

Pero que el sistema funcione bien depende de que esas decisiones se cumplan en el código y de que los requisitos se prueben. Buena parte ya está implementada: adaptadores y repositorios que aíslan el dominio, eventos que avisan a los contactos cuando cambia una alerta, tareas programadas que actúan aunque el celular pierda la señal y colas en la app para no perder ubicaciones ni evidencias. Lo que falta es probar y documentar cómo se comporta todo esto en la práctica, sobre todo en alertas, ubicación, contactos y protección de la información. El SRS sirvió de guía; que el proyecto cumpla sus objetivos se demuestra con el sistema funcionando y validado.

REFERENCIAS

[1] Instituto Nacional de Salud, *Informe de Evento 2023: Violencia de Género e Intrafamiliar y Ataques con Agentes Químicos*. Bogotá, Colombia: INS. [Online]. Available: https://www.ins.gov.co/buscador-eventos/Informesdeevento/VIOLENCIA%20DE%20GENERO%20INFORME%20DE%20EVENTO%202023.pdf

[2] ONU Mujeres Colombia, “ONU Mujeres y el DANE presentan la tercera edición del estudio Mujeres y hombres: brechas de género en Colombia, evidenciando persistentes desigualdades en el país,” 14 nov. 2024. [Online]. Available: https://colombia.unwomen.org/es/stories/noticia/2024/11/onu-mujeres-y-el-dane-presentan-la-tercera-edicion-del-estudio-mujeres-y-hombres-brechas-de-genero-en-colombia-evidenciando-persistentes-desigualdades-en-el-pais (accedido: 13 oct. 2026).

[3] M. Sumra, S. Asghar, K. S. Khan, J. M. Fernández-Luna, J. F. Huete, and A. Bueno-Cavanillas, “Smartphone apps for domestic violence prevention: A systematic review,” *Int. J. Environ. Res. Public Health*, vol. 20, no. 7, Art. no. 5246, 2023, doi: 10.3390/ijerph20075246.

[4] K. Ford, M. A. Bellis, N. Judd, N. Griffith, and K. Hughes, “The use of mobile phone applications to enhance personal safety from interpersonal violence – an overview of available smartphone applications in the United Kingdom,” *BMC Public Health*, vol. 22, Art. no. 1158, 2022, doi: 10.1186/s12889-022-13551-9.

[5] A. De la Cruz Villarreal, G. N. Alavéz Santiago, Á. Martínez, E. L. Baltazar Hernández, J. C. López Pérez, and M. R. Peralta Calvo, “SAM: Interface design of a mobile application for women in risk situations,” in *Ext. Abstr. CHI Conf. Human Factors Comput. Syst. (CHI EA '23)*, 2023, pp. 1–6, doi: 10.1145/3544549.3583844.

[6] A. Lakshmipathy and T. Kumar, “A smartphone application for women's safety,” in *Navigating the Technological Tide: The Evolution and Challenges of Business Model Innovation* (Lecture Notes in Networks and Systems). Cham, Switzerland: Springer, 2024, pp. 197–205, doi: 10.1007/978-3-031-67444-0_19.

[7] K. Childs et al., “‘I had sort of a sense that I was always being watched...since I was’: Examining interpersonal discomfort from continuous location-sharing applications,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS '24)*, Salt Lake City, UT, USA, 2024, pp. 4197–4211, doi: 10.1145/3658644.3690342.

[8] Y. Ma, C. Sew, Z. Sarsenbayeva, J. Knibbe, and J. Goncalves, “Understanding users' perspectives on location privacy management on iPhones,” *Proc. ACM Hum.-Comput. Interact.*, vol. 8, no. MHCI, pp. 1–25, 2024, doi: 10.1145/3676529.

[9] Congreso de la República de Colombia, Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales, oct. 2012. [Online]. Available: https://www.suin-juriscol.gov.co/viewDocument.asp?id=30019730

[10] Android Developers, “Access location in the background.” [Online]. Available: https://developer.android.com/develop/sensors-and-location/location/background (accedido: 13 oct. 2026).

[11] Android Developers, “Foreground service types.” [Online]. Available: https://developer.android.com/develop/background-work/services/fgs/service-types (accedido: 13 oct. 2026).

[12] J. Kim, N. Chang, and D. Shin, “Mobile GPS application design based on system-level power and battery status estimation,” *Energies*, vol. 14, no. 17, Art. no. 5333, 2021, doi: 10.3390/en14175333.

[13] B. Thierry, K. Stanley, Y. Kestens, M. Winters, and D. Fuller, “Comparing location data from smartphone and dedicated global positioning system devices: Implications for epidemiologic research,” *Am. J. Epidemiol.*, vol. 193, no. 1, pp. 180–192, 2024, doi: 10.1093/aje/kwad176.

[14] Y. Abgaz et al., “Decomposition of monolith applications into microservices architectures: A systematic review,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4213–4242, 2023, doi: 10.1109/TSE.2023.3287297.

[15] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning, 2018.

[16] R. Su and X. Li, “Modular monolith: Is this the trend in software architecture?” in *Proc. 1st Int. Workshop New Trends Softw. Archit.*, 2024, pp. 10–13, doi: 10.1145/3643657.3643911.

[17] T. R. H. Barzotto and K. Farias, “Evaluation of the impacts of decomposing a monolithic application into microservices: A case study,” arXiv:2203.13878, 2022.

[18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF, RFC 7519, May 2015.

[19] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” IETF, RFC 8725, Feb. 2020.

[20] OWASP Foundation, “OWASP Application Security Verification Standard (ASVS),” [versión por confirmar]. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[21] OWASP Foundation, “OWASP Mobile Application Security Verification Standard (MASVS),” [versión por confirmar]. [Online]. Available: https://mas.owasp.org/MASVS/